Split-Version Documentation

User guide, architecture reference, feature map, and file-by-file technical manual

This edition consolidates the previous split-version documents into one continuous manual. It keeps all distinct ideas, removes repeated explanations, and organizes the material in a single order that is easier to read and maintain.

How this manual is organized. Part I explains how to use the application. Part II explains how it is built. User-facing sections avoid source-code detail unless it helps explain behavior. Technical sections avoid re-explaining user workflow and instead focus on architecture, files, functions, and feature implementation.

Part I — User Guide

1. What the application is

The split version is a desktop image-editing and color-grading application for PNG, JPG, and JPEG files. It follows a non-destructive workflow: the source image stays intact in memory, the active edit is stored as state, and the visible image is rebuilt from that state whenever the session changes.

The application is designed for responsive interactive work rather than one-shot filter application. Broad tonal correction, per-channel shaping, region-based color work, curves, histogram monitoring, crop, rotate, flip, resize, preset reuse, and project save/load are all part of the same editing flow.

2. Libraries used and why they matter

Library	Used for	Why it matters
PySide6	Desktop interface, custom widgets, dialogs, actions, timers, graphics-view display, signals, threads	Provides the full UI layer and the event-driven behavior of the application.
NumPy	RGBA image arrays, LUT application, channel math, tonal masks, clipping, histograms, preview hashing	Acts as the core image-processing data layer.
Pillow (PIL)	Image load/save, RGBA normalization, fallback resizing	Handles reliable file I/O and provides a fallback when OpenCV is unavailable.
OpenCV (cv2)	Fast resizing and fast LUT application when available	Improves responsiveness for live preview on larger images.
dataclasses	Curves, tones, crop, resize, session state	Keeps the state model explicit and easy to serialize.
json	Preset and project save/load	Makes session data portable and easy to persist.

3. Core editing model

- The original image is normalized to a contiguous RGBA NumPy array when it is loaded.
- The active edit is represented by AdjustmentState and related dataclasses, not by repeatedly overwriting the source bitmap.
- Undo and redo store state snapshots instead of full image copies.
- The preview is rendered from the original image plus the current state.
- A fast preview pass is used during live interaction, and a full-quality pass is requested after the interaction settles.
- Histogram refresh is tied to the full-quality pass so the interface stays more responsive during drag operations.

4. Main capabilities

- Open images in PNG, JPG, and JPEG format.
- Export edited results as PNG or JPEG, with warnings for lossy JPEG output and transparency loss.
- Adjust brightness, contrast, gamma, exposure, temperature, tint, white-balance strength, and per-channel intensity.
- Apply shadows, midtones, and highlights RGB adjustments.
- Edit separate curves for master, red, green, and blue.

- View RGB histograms in both a dedicated panel and inside the curve editor background.
- Crop interactively with drag handles and optional aspect-ratio lock.
- Rotate, flip, resize, zoom, fit to window, and compare before/after.
- Save and load presets and project files.
- Use drag-and-drop to open images.

5. Interface layout

Area	Purpose
Toolbar	Quick access to open, export, undo, redo, reset, project, preset, fit, and before/after actions.
Adjustments tab	Main grading controls, channel intensity, tonal-region controls, and reset actions.
Curves tab	Channel selector plus the interactive curve editor.
Histogram tab	Larger RGB histogram panel and a short note about histogram refresh behavior.
Transform tab	Navigation, crop, rotate, flip, and resize controls.
Viewer	Displays the preview image and handles zoom, pan, crop editing, and drag-and-drop loading.

6. Typical editing workflow

1. Open an image with Ctrl+O, the toolbar action, or drag-and-drop into the viewer.
2. Start with broad tonal and balance corrections such as exposure, contrast, gamma, temperature, tint, and white-balance strength.
3. Use channel intensity and shadows, midtones, and highlights controls when the image needs more selective color shaping.
4. Refine the result with curves after the broad corrections are already close.
5. Check the histogram and curve overlay while watching for clipping or channel imbalance.
6. Apply crop, rotate, flip, and resize once the color work is close to final.
7. Compare the edited view to the original before exporting or saving the session.
8. Export the image, or save a preset or project if you plan to continue later.

7. Preview model and responsiveness

The application uses a two-stage preview strategy. During interaction, it favors responsiveness. After interaction ends, it favors accuracy.

- When the user drags a slider or a curve point, the application marks the interaction as live and queues a debounced fast render.
- Fast renders reduce unnecessary work during micro-movements and keep the interface more fluid.
- A second timer requests a full-quality render once the user pauses.
- The full-quality pass is also the pass that refreshes histogram data.
- Rendering is handled by a worker object that lives on a separate QThread, which keeps the UI thread from doing the full pipeline synchronously.
- A render generation counter prevents stale results from older requests from overwriting newer preview states.

8. File operations

Open image. Loads a new source image, resets the state, clears prior history, refreshes the preview, and reinitializes geometry-sensitive controls.

Export image. Applies crop, rotate, flip, resize, and color adjustments to the source image and writes the final result to disk.

Save preset. Stores adjustment and geometry parameters, but not the source image itself.

Load preset. Restores a saved parameter state and applies it to the currently open image.

Save project. Stores the source image path plus the full parameter state so editing can continue later.

Load project. Reopens the saved source image and restores the parameter state from the project file.

9. Save naming behavior

When a target filename already exists, the application avoids destructive overwrite-by-default behavior. Instead, it can suggest a unique path such as image(1).png, image(2).png, and so on. The numbering is based on the actual contents of the target directory rather than a temporary in-memory counter.

10. Color controls in practical terms

Brightness. Applies a uniform tonal offset through the scalar LUT path; useful for quick overall lightening or darkening.

Contrast. Expands or compresses separation around the tonal midpoint.

Gamma. Changes the shape of the tonal response, especially in the midtones.

Exposure. Acts like a multiplicative light shift and is useful for broad correction before finer shaping.

Temperature. Biases the image warmer or cooler by shifting red and blue response in opposite directions.

Tint. Shifts the image along the green–magenta axis.

White-balance strength. Controls how strongly the white-balance correction influences the image.

Channel intensity. Applies extra channel-specific gain after the LUT stage.

Shadows / midtones / highlights. Apply RGB offsets in tonal bands rather than across the entire image, which is useful for more selective color shaping.

11. Curves and histogram behavior

- The curve editor works in normalized 0–1 space and converts the final curve into a 256-entry lookup table.
- Curve endpoints are constrained so the graph always spans the full input range.
- Interior points can be added with left-click and removed with right-click.
- During dragging, point ordering is preserved so the curve remains monotonic along the x-axis.
- The histogram drawn behind the curve is channel-aware: master mode can show all RGB channels, while single-channel mode emphasizes the selected channel.
- The dedicated Histogram tab shows a larger standalone RGB histogram for easier inspection.

12. Crop, rotate, flip, and resize

Geometry is handled carefully so the image the user sees and the image the processing pipeline uses stay as aligned as possible.

- Crop rectangles are interpreted in transformed space, meaning after rotation and flipping but before crop is actually applied.
- When rotation or flipping changes, crop coordinates are remapped so the crop continues to target the same underlying region as closely as possible.
- Resize is treated as a later stage than transform and crop, so its base image is the already transformed image.
- When a 90-degree or 270-degree rotation is applied to an already resized state, width and height are swapped so output orientation stays sensible.
- Crop aspect lock can constrain the crop box to ratios such as Original, 1:1, 4:3, 3:2, 16:9, or 21:9.

13. Undo, redo, reset, and comparison

- Undo and redo are parameter-based rather than image-copy-based, which reduces memory pressure.
- Reset All clears the full adjustment state.
- Reset Current Tab clears only the category represented by the active tab.
- Narrower reset actions also exist for one channel, one tonal group, the current curve, crop, and resize.
- Before/after comparison uses the original image versus the edited preview, rather than a second independently processed branch.

14. Shortcuts

Shortcut	Action
Ctrl+O	Open image
Ctrl+S	Export image
Ctrl+R	Reset all
Ctrl+Z	Undo
Ctrl+Y	Redo
+	Zoom in
-	Zoom out
Space	Toggle or temporarily preview the original, depending on the active path

15. Troubleshooting and limitations

- JPEG export does not preserve transparency; the application warns about that before saving.
- Performance differs depending on whether OpenCV is installed, because resize and LUT operations are faster with OpenCV.
- Undo history is state-snapshot based rather than node-graph based; history can still grow if many distinct states are pushed.
- Because geometry and preview transforms are complex, edge cases can still appear around combined crop, rotate, flip, and resize workflows.

Part II — Architecture and Technical Reference

16. Split-package structure

In the split version, the code is divided into small files with clearer responsibilities instead of one large monolithic script. That makes the application easier to navigate, maintain, and extend.

File	Role	Why it exists
config.py	Shared constants	Keeps filters, render timings, preview sizing, and other common constants in one place.
utils.py	General helpers	Holds conversion, resize, histogram, LUT, save-path, and aspect-fit helpers.
transforms.py	Coordinate mapping	Keeps crop geometry stable across rotation and flips.
models.py	Session/state definitions	Defines dataclasses for curves, tones, crop, resize, adjustment state, and history.
processing.py	Image pipeline	Applies geometry, resize, and color work to image arrays.
render.py	Worker-side rendering	Moves preview rendering out of the busiest part of the UI thread.
widgets/curve_editor.py	Curve widget	Interactive curve editing with histogram overlay.
widgets/histogram_widget.py	Histogram widget	Draws a standalone histogram panel.
widgets/image_view.py	Image canvas widget	Handles display, crop overlay, zoom, pan, and drag-and-drop.
main_window.py	Application controller	Connects widgets, state, persistence, history, and rendering.
main.py / __main__.py	Startup entry points	Create QApplication and launch the main window.

17. Architecture at a glance

- The original image is loaded once and stored as a contiguous RGBA NumPy array.
- The current edit is represented by AdjustmentState and related dataclasses.
- Undo and redo store state snapshots, not full image copies.
- The preview is rendered from the original image plus the current state.
- The viewer shows a fitted preview, while export uses the full processed result.
- Fast preview requests are debounced, and full-quality renders are triggered after interaction settles.

18. Practical mental model

The easiest way to understand the codebase is as four layers.

Layer	Contents
State	models.py dataclasses and history
Processing	utils.py helpers, transforms.py geometry conversion, processing.py image pipeline, render.py worker
Widgets	curve editor, histogram widget, image view

Controller	main_window.py plus startup entry point

19. Full processing pipeline

9. Open the source image and normalize it to RGBA.
10. Apply rotation.
11. Apply horizontal and vertical flips.
12. Apply crop in transformed space.
13. Apply explicit resize if enabled.
14. Apply the scalar grading LUT for brightness, contrast, gamma, and exposure.
15. Apply the master curve and per-channel curves.
16. Apply temperature, tint, white-balance strength, and RGB intensity multipliers.
17. Apply shadows, midtones, and highlights adjustments.
18. Fit the result to the preview viewport for display.

20. Major feature implementation map

Feature	Implemented by	How it works
Open image	main_window.py: open_image, load_image; utils.py: pil_to_numpy_rgba	Loads the file, resets state/history, and rebuilds preview and histogram.
Export image	main_window.py: export_image; processing.py; utils.py: choose_save_path, unique_path	Processes the current state at export quality and writes PNG or JPEG output.
Presets	main_window.py: save_preset, load_preset; models.py: AdjustmentState.to_json/from_json	Saves and restores adjustment parameters only.
Projects	main_window.py: save_project, load_project	Stores the source image path plus the full adjustment state.
Undo / redo	models.py: HistoryManager; main_window.py: commit_state, undo, redo, finalize_interaction	Uses state snapshots rather than full image buffers.
Live preview	main_window.py: request_fast_render, request_full_render; render.py: RenderWorker	Uses debounced fast passes and full-quality idle passes.
Curves	widgets/curve_editor.py; utils.py: build_curve_lut; processing.py: apply_color	Turns curve points into LUTs and applies them per channel.
Histogram	utils.py: histogram_from_rgba; widgets/histogram_widget.py; curve editor overlay	Builds RGB histograms and draws them in standalone and embedded views.
Crop	widgets/image_view.py; transforms.py; main_window.py: apply_crop_from_view	Edits crop in viewer space and converts it back to processing space.
Resize	main_window.py: on_resize_dimension_changed, on_resize_controls_changed; processing.py: apply_resize	Uses pre-resize geometry as the base reference for dimension locking.
Rotate / flip	main_window.py: rotate_image, toggle_flip; processing.py: apply_crop_rotate_flip; transforms.py	Keeps transforms and crop mapping consistent.

21. File-by-file technical reference

21.1 config.py

This file contains configuration constants used across the package. It does not contain executable logic, but it is important because it centralizes values that would otherwise be duplicated.

- APP_TITLE controls the application title.
- SUPPORTED_INPUT and SUPPORTED_EXPORT define file-dialog filters.
- PROJECT_FILTER and PRESET_FILTER define project and preset file types.
- FULL_IDLE_DELAY_MS and RENDER_DEBOUNCE_MS control rendering responsiveness.
- HISTOGRAM_MAX_SIDE limits histogram work size for performance.
- PREVIEW_RENDER_SCALE slightly oversamples the preview for better visual sharpness.
- MIN_CROP_SIZE prevents unusably small crop boxes.

21.2 utils.py

This file contains general-purpose helper functions shared by both the UI layer and the processing layer.

pil_to_numpy_rgba(img)

- What it does: Converts a Pillow image into the application's internal RGBA NumPy format.
- How it works: It forces RGBA mode, converts the pixels to a uint8 NumPy array, and makes the array contiguous in memory so later processing and Qt conversion are stable.
- Feature role: Used at image-load time.

numpy_to_qimage(arr)

- What it does: Converts a NumPy image array into a QImage for display.
- How it works: It inspects the channel count, chooses the matching Qt image format, builds a QImage over the underlying memory, and then copies it so the Qt object remains valid if the Python array changes later.
- Feature role: Used before display in Qt widgets.

numpy_to_pixmap(arr)

- What it does: Builds a QPixmap from a NumPy array.
- How it works: It is a small wrapper around numpy_to_qimage followed by QPixmap.fromImage.
- Feature role: Used by the image viewer scene.

resize_rgba(arr, size, fast=True)

- What it does: Resizes an RGBA image array to a target width and height.
- How it works: It skips work when the target size already matches the input. If OpenCV is available it uses cv2.resize with fast or high-quality interpolation. Otherwise it falls back to Pillow resizing.
- Feature role: Used by explicit resize, preview fit, and internal downscales.

downscale_rgba(arr, max_side)

- What it does: Shrinks an image so its longest side does not exceed a limit.
- How it works: It measures the current longest dimension, computes a scale factor only when needed, and then calls resize_rgba with the reduced size.
- Feature role: Used mainly before histogram calculation.

hash_array(arr)

- What it does: Creates a lightweight preview identity token.

- How it works: Instead of hashing every pixel, it samples the array on a coarse grid and sums sampled RGB values. That makes it cheap enough for UI refresh checks.
- Feature role: Prevents redundant pixmap refreshes.

fit_size_preserving_aspect(src_w, src_h, max_w, max_h)

- What it does: Computes a fitted width and height while preserving aspect ratio.
- How it works: It clamps minimum dimensions, computes the width and height scale factors, takes the limiting one, and returns rounded integer dimensions.
- Feature role: Used for preview fitting and crop-space conversions.

unique_path(path)

- What it does: Finds the next available non-conflicting filename in a directory.
- How it works: It inspects the target directory, recognizes siblings with the same stem and extension plus numeric suffixes like (1) and (2), and returns the next free path.
- Feature role: Used by export, preset save, and project save.

choose_save_path(parent, title, suggested, file_filter)

- What it does: Opens a save dialog that already suggests a unique filename.
- How it works: It turns the suggested path into an absolute path, computes the next free name with unique_path, pre-fills the dialog, and disables overwrite confirmation so the application can keep its own naming behavior.
- Feature role: Used by file-save workflows.

build_curve_lut(points)

- What it does: Converts curve control points into a 256-entry lookup table.
- How it works: It clamps points to the 0–1 domain, sorts them by x-value, guarantees the endpoints exist, interpolates across 256 samples, and maps the result into uint8 output values.
- Feature role: Used by the curve-processing stage.

compose_scalar_lut(brightness, contrast, gamma, exposure)

- What it does: Creates the lookup table used for global scalar tonal adjustments.
- How it works: It starts from normalized values, applies exposure scaling, brightness shift, contrast around the midpoint, and gamma correction, then maps back into 0–255 integer output values.
- Feature role: Used by the scalar grading stage.

histogram_from_rgba(arr)

- What it does: Builds red, green, and blue histograms from an image array.
- How it works: It first downsizes the image for efficiency, strips alpha, and runs NumPy histogram calculations separately for each RGB channel.
- Feature role: Used by the histogram displays.

21.3 transforms.py

This file isolates transform and coordinate-conversion math used to keep crop behavior stable.

transformed_size(width, height, rotation)

- What it does: Computes the dimensions of an image after rotation.
- How it works: For 90-degree and 270-degree rotations width and height swap. For 0-degree and 180-degree rotations they stay in the same order.
- Feature role: Used by geometry conversion helpers.

transform_point_original_to_view(...)

- What it does: Maps one point from original-image space into transformed view space.
- How it works: It applies rotation first, tracks the transformed bounds, and then applies optional horizontal and vertical flips inside that transformed coordinate system.
- Feature role: Used when preserving crop meaning across transforms.

inverse_transform_point_view_to_original(...)

- What it does: Maps one point from transformed view space back into original-image space.
- How it works: It reverses flips first and then applies the inverse mapping for the selected rotation.
- Feature role: Used for reverse coordinate conversion.

rect_view_to_original(...)

- What it does: Converts a rectangle from transformed-view coordinates into original-image coordinates.
- How it works: It maps the rectangle corners with the inverse point transform, computes the bounding box of the mapped points, and clamps the result to valid original-image bounds.
- Feature role: Used when remapping crop content.

rect_original_to_view(...)

- What it does: Converts a rectangle from original-image coordinates into transformed-view coordinates.
- How it works: It transforms each corner into rotated and flipped space, builds a bounding box, and clamps the result to valid transformed bounds.
- Feature role: Used when restoring crop after rotate or flip changes.

21.4 models.py

This file defines the data structures that represent the editable session and its history.

- CurveSet stores the four curve definitions used by the color pipeline.
- ToneRGB stores one three-channel tonal adjustment group.
- CropRect stores crop geometry and whether cropping is active.
- ResizeState stores resize dimensions and whether resize is active.
- AdjustmentState gathers scalar grading, tonal controls, geometry state, crop, resize, and curves into one object.
- HistoryManager manages undo and redo stacks of AdjustmentState snapshots.

CurveSet.to_json()

- What it does: Serializes all four curves.
- How it works: It converts point tuples into plain lists so the data can be written to JSON-based preset and project files.
- Feature role: Used by project and preset save.

CurveSet.from_json(data)

- What it does: Rebuilds a CurveSet from stored data.
- How it works: It restores each channel's point list and falls back to identity curves when data is missing.
- Feature role: Used by project and preset load.

ToneRGB.to_json() / from_json(data)

- What it does: Serializes and restores one tonal RGB block.
- How it works: It converts the dataclass to a plain dictionary and back again.
- Feature role: Used for shadows, midtones, and highlights.

CropRect.to_json() / from_json(data)

- What it does: Serializes and restores crop state.

- How it works: It stores x, y, width, height, and the enabled flag.
- Feature role: Used for project and preset persistence.

ResizeState.to_json() / from_json(data)

- What it does: Serializes and restores resize state.
- How it works: It stores width, height, and the enabled flag.
- Feature role: Used for project and preset persistence.

AdjustmentState.clone()

- What it does: Creates a deep copy of the full session state.
- How it works: It uses deepcopy so nested dataclasses such as curves and tonal controls are copied safely without shared references.
- Feature role: Used for history and temporary edits.

AdjustmentState.to_json() / from_json(data)

- What it does: Serializes and restores the full session state.
- How it works: It bundles scalar, tonal, geometry, crop, resize, and curve values into one structure.
- Feature role: Used by projects and presets.

HistoryManager.push(state)

- What it does: Adds a new state to undo history when it differs from the latest one.
- How it works: It compares serialized state contents to avoid duplicate consecutive snapshots and clears redo when a new branch is created.
- Feature role: Implements non-destructive history.

HistoryManager.undo(current) / redo(current)

- What it does: Moves backward or forward through state history.
- How it works: Undo moves the latest state to redo and returns the previous one; redo performs the reverse operation.
- Feature role: Implements undo and redo.

21.5 processing.py

This file contains the actual image-processing stages used by preview and export.

ImageProcessor.apply_crop_rotate_flip(img, state)

- What it does: Applies geometry transforms before the color stage.
- How it works: It rotates first, then flips, then applies the crop rectangle. This order keeps crop coordinates aligned with the same transformed space the user sees in the viewer.
- Feature role: Implements rotate, flip, and crop.

ImageProcessor.apply_resize(img, state, fast)

- What it does: Applies the optional resize stage.
- How it works: It checks whether resize is enabled and whether width and height are valid, then calls the shared resize helper.
- Feature role: Implements explicit resize.

ImageProcessor.apply_color(img, state, skip_tonal=False)

- What it does: Applies the color-grading pipeline.
- How it works: It builds scalar and curve LUTs, applies them to the RGB image, converts to float space, applies white balance, tint, per-channel intensity, and optional shadows, midtones, and highlights adjustments, then converts back to uint8 RGBA.

- Feature role: Implements grading controls, curves, tonal controls, and white-balance logic.

21.6 render.py

This file contains the worker-side rendering classes.

RenderRequest.__init__(...)

- What it does: Stores everything needed for one render pass.
- How it works: It captures the generation number, source image, adjustment state, target display size, quality mode, and whether tonal work may be skipped.
- Feature role: Keeps rendering requests explicit and self-contained.

RenderWorker.submit(request)

- What it does: Queues a render request.
- How it works: It stores only the newest pending request and starts processing immediately if the worker is idle.
- Feature role: Helps keep live preview responsive.

RenderWorker._process_next()

- What it does: Processes the next pending request.
- How it works: It applies geometry, resize, preview fitting, and color grading. It then emits the rendered image, and for full-quality passes it also emits histogram data. If a newer request arrived while rendering, it continues with that one immediately.
- Feature role: Drives the preview pipeline.

21.7 widgets/histogram_widget.py

This file contains the standalone histogram display widget.

HistogramWidget.set_histogram(hist)

- What it does: Loads histogram data into the widget.
- How it works: It stores the histogram dictionary and calls update so paintEvent will redraw the graph.
- Feature role: Used by the Histogram tab.

HistogramWidget.paintEvent(event)

- What it does: Draws the histogram panel contents.
- How it works: It paints the background, border, scales channel values to the current maximum, and draws filled area graphs for each RGB channel.
- Feature role: Implements the histogram panel.

21.8 widgets/curve_editor.py

This file contains the interactive curve editor widget.

CurveEditor.set_channel(channel)

- What it does: Changes which curve is displayed and edited.
- How it works: It stores the selected channel and repaints the widget.
- Feature role: Implements master, red, green, and blue switching.

CurveEditor.set_points(points)

- What it does: Loads a new set of control points.
- How it works: It sorts the list by x-position and schedules a redraw.
- Feature role: Keeps the curve display synchronized with state.

CurveEditor.set_histogram(hist)

- What it does: Supplies histogram data for the curve background.
- How it works: It stores the histogram arrays and repaints so the background reflects the latest tonal distribution.
- Feature role: Lets the curve editor show tonal context.

CurveEditor._content_rect()

- What it does: Computes the square graph area inside the widget.
- How it works: It takes the smaller of the available width and height after margins so the curve plot remains square.
- Feature role: Improves visual consistency.

CurveEditor._to_widget(p) and _to_normalized(pos)

- What it does: Convert between normalized curve space and widget space.
- How it works: One maps logical points to pixels, the other maps mouse positions back into 0–1 coordinates.
- Feature role: Support interactive editing.

CurveEditor._find_handle(pos)

- What it does: Detects whether the mouse is near an existing point handle.
- How it works: It checks proximity against the rendered point handles.
- Feature role: Used for drag and delete logic.

CurveEditor.mousePressEvent / mouseMoveEvent / mouseReleaseEvent

- What it does: Handle point insertion, dragging, and drag finalization.
- How it works: Left-click selects or inserts, right-click removes a non-endpoint, move updates the point while preserving order, and release emits dragFinished.
- Feature role: Implements interactive curve editing.

CurveEditor.paintEvent(event)

- What it does: Draws the curve editor contents.
- How it works: It paints the background, histogram overlay, grid, border, active curve line, and draggable point handles using the color associated with the selected channel.
- Feature role: Implements the visual curve tool.

21.9 widgets/image_view.py

This file contains the image-canvas widget responsible for display and crop interaction.

- It is built on QGraphicsView and QGraphicsScene.
- It displays the preview pixmap.
- It supports zoom and panning.
- It implements drag-and-drop image opening.
- It draws the crop overlay, handles, and guides.
- It manages staged crop editing before the crop is committed to application state.

ImageView.set_image(pixmap)

- What it does: Displays a new pixmap in the scene.
- How it works: It replaces the current pixmap, updates the scene rect, and repaints the viewport.
- Feature role: Updates the visible preview.

ImageView.fit_image(), zoom_in(), zoom_out()

- What it does: Control view magnification.
- How it works: fit_image uses fitInView with preserved aspect ratio; zoom_in and zoom_out scale the graphics-view transform.
- Feature role: Implement viewer navigation.

ImageView.set_crop_mode(enabled)

- What it does: Turns crop-editing mode on or off.
- How it works: It switches between crop interaction and normal hand-drag panning.
- Feature role: Implements crop mode.

ImageView.set_crop_lock(enabled, ratio)

- What it does: Stores aspect-lock behavior for crop editing.
- How it works: The ratio is later used when corners or edges are dragged under lock mode.
- Feature role: Implements locked crop ratios.

ImageView.current_crop_rect(), commit_staged_crop(), revert_staged_crop()

- What it does: Manage staged crop state.
- How it works: These methods expose the staged crop rectangle, commit it, or restore the previous committed version.
- Feature role: Separate preview editing from final crop commit.

ImageView.dragEnterEvent / dragMoveEvent / dropEvent

- What it does: Accept supported image files dropped into the viewer.
- How it works: The widget checks file extensions and emits imageDropped when a valid image is released.
- Feature role: Implements drag-and-drop loading.

ImageView.drawForeground(...)

- What it does: Draws the crop overlay on top of the image.
- How it works: It darkens the outside region, outlines the crop rectangle, draws rule-of-thirds guides, and paints edge, corner, and center handles sized against zoom.
- Feature role: Implements crop visualization.

ImageView._pick_handle(...), _clamp_crop(...), _apply_aspect_to_corner(...), _update_crop_from_handle(...)

- What it does: Handle crop interaction math.
- How it works: These helpers identify the dragged handle, keep the crop box valid, preserve aspect ratio when requested, and update crop geometry as the mouse moves.
- Feature role: Implement the crop editing mechanics.

ImageView.mousePressEvent / mouseMoveEvent / mouseReleaseEvent

- What it does: Handle crop interaction lifecycle.
- How it works: Press chooses or creates a crop handle interaction, move updates the crop box live, and release finishes the drag.
- Feature role: Implement direct crop manipulation.

21.10 main_window.py

This is the application's main controller. It coordinates UI construction, state synchronization, render scheduling, feature actions, file operations, history, and transform consistency.

- It owns the current image, current state, history, worker thread, timers, and main widgets.

- It is the bridge between user interaction and the rendering pipeline.
- It turns slider movement and widget signals into state changes and preview requests.

MainWindow.__init__()

- What it does: Initializes the application state, worker, timers, and UI.
- How it works: It sets up internal state, creates the background render worker, connects signals, then builds and styles the interface.
- Feature role: Application startup.

MainWindow.build_ui()

- What it does: Constructs the main window.
- How it works: It creates the splitter, tab widget, image viewer, actions, toolbar, and each control tab.
- Feature role: Creates the visible UI.

MainWindow.apply_styles()

- What it does: Applies the application's shared dark theme.
- How it works: It uses a Qt stylesheet to define colors, borders, padding, and tab styling.
- Feature role: Gives the application a consistent look.

MainWindow.build_adjust_tab()

- What it does: Builds the grading controls tab.
- How it works: It creates grouped sliders for scalar grading, per-channel intensity, tonal controls, and reset actions.
- Feature role: Implements the main edit panel.

MainWindow.build_curve_tab()

- What it does: Builds the curve-editing tab.
- How it works: It adds the channel selector, reset button, and CurveEditor widget.
- Feature role: Implements curve editing UI.

MainWindow.build_histogram_tab()

- What it does: Builds the histogram tab.
- How it works: It adds the standalone histogram widget and descriptive text.
- Feature role: Implements histogram viewing.

MainWindow.build_transform_tab()

- What it does: Builds the transform tools tab.
- How it works: It adds zoom controls, rotate and flip buttons, crop controls, and resize controls.
- Feature role: Implements geometry tools.

MainWindow.create_actions() and create_toolbar()

- What it does: Build actions and the top toolbar.
- How it works: They define keyboard shortcuts, reusable QAction objects, and toolbar layout.
- Feature role: Provide quick access to key features.

MainWindow.get_display_target_size()

- What it does: Computes preview render dimensions.
- How it works: It starts from the transformed image size, considers explicit resize dimensions if active, and then fits the preview inside the viewer viewport.
- Feature role: Used before each render request.

MainWindow.get_crop_reference_size()

- What it does: Computes the transformed image size before crop is applied.
- How it works: It clones the state, temporarily disables crop, and processes rotation and flips only. That gives a stable coordinate space for crop editing.
- Feature role: Keeps crop editing consistent.

MainWindow.geometry_rect_to_display_rect() and display_rect_to_geometry_rect()

- What it does: Convert crop rectangles between processing space and viewer space.
- How it works: They scale rectangles based on the transformed reference image size and current preview size.
- Feature role: Connect crop editing to actual image processing.

MainWindow.request_fast_render(), flush_fast_render_request(), request_full_render()

- What it does: Schedule preview work.
- How it works: Fast renders are debounced for responsiveness, while full renders are requested after interaction settles and also refresh histogram data.
- Feature role: Implement responsive live preview.

MainWindow.update_viewer_pixmap()

- What it does: Refreshes the preview shown in the viewer.
- How it works: It uses a lightweight content hash to skip unnecessary pixmap rebuilds and optionally fits the image after load.
- Feature role: Keeps UI updates efficient.

MainWindow.apply_histogram_to_widgets()

- What it does: Pushes histogram data into both histogram displays.
- How it works: It updates the standalone histogram widget and the histogram overlay used in the curve editor.
- Feature role: Synchronizes histogram views.

MainWindow.confirm_discard_unsaved()

- What it does: Prompts the user about unsaved changes.
- How it works: It offers Save Project, Discard, or Cancel before closing when the session is dirty.
- Feature role: Implements session-safety behavior.

MainWindow.on_slider_changed(), on_tone_slider_changed(), on_curve_points_changed()

- What it does: Handle live edits from controls.
- How it works: Each method clones the current state, updates only the changed field, and commits the new state without immediately finalizing history.
- Feature role: Implement live editing.

MainWindow.apply_crop_from_view() and cancel_crop_edit()

- What it does: Commit or discard staged crop changes.
- How it works: The crop box is edited visually in the viewer and only converted back into geometry state when the user clicks Apply Crop.
- Feature role: Implement crop commit workflow.

MainWindow.on_resize_dimension_changed() and on_resize_controls_changed()

- What it does: Handle resize interaction and aspect locking.
- How it works: They compute the true pre-resize base image, preserve aspect ratio when requested, and write the resize values back into state.
- Feature role: Implement resize behavior.

MainWindow.undo(), redo(), reset_all(), reset_current_tab()

- What it does: Manage history and resets.
- How it works: Undo and redo restore saved AdjustmentState snapshots; resets build new clean state variants and commit them.
- Feature role: Implement edit recovery and reset tools.

MainWindow.remap_crop_for_new_transform()

- What it does: Keeps crop attached to the same visual content after rotate or flip.
- How it works: It converts the crop rectangle back into original-image space and then forward into the new transformed space.
- Feature role: Preserves crop meaning across transforms.

MainWindow.remap_resize_for_new_transform()

- What it does: Keeps resize orientation consistent across rotation.
- How it works: When the image rotates by 90° or 270°, width and height are swapped so a stretched image stays stretched in the correct direction.
- Feature role: Preserves resize meaning across orientation changes.

MainWindow.rotate_image() and toggle_flip()

- What it does: Apply transforms while preserving crop and resize logic.
- How it works: They update transform state, remap dependent crop and resize values, and commit the result.
- Feature role: Implement rotate and flip tools.

MainWindow.toggle_before_after()

- What it does: Switches between original and edited preview.
- How it works: It toggles a boolean and repaints the viewer with either the preview image or the original image.
- Feature role: Implements before/after comparison.

21.11 Startup entry points

install_slider_commit_hooks(window)

- What it does: Attaches slider press and release behavior to interaction finalization.
- How it works: It marks the beginning of a live drag on press and calls finalize_interaction on release so history behaves cleanly.
- Feature role: Improves slider UX and history consistency.

main()

- What it does: Starts the application.
- How it works: It creates QApplication, creates MainWindow, installs the slider commit hooks, shows the window, and enters the Qt event loop.
- Feature role: Program entry point.